13/7/2026

Project 1: Detecting Patterns in Tabular Medical Data with MIMIC-III

Introduction

This project is designed to introduce the intricacies of healthcare data analysis using the MIMIC-III dataset. Participants will apply classical machine learning models and explore the basics of neural networks to gain insights into patient data. The project blends theoretical knowledge with practical application.

You will learn to preprocess and analyze healthcare data, starting with basic statistical methods and data distributions. You will utilize Python libraries such as NumPy, Pandas, Matplotlib, and scikit-learn to manipulate data and visualize results. Also, you will start using TensorFlow (Keras) for deep learning model training. The project will cover classical supervised learning models including linear regression, decision trees, and support vector machines (SVM). Also, implementing a simple feedforward neural network.

Through hands-on tutorials, you will learn to evaluate model performance using various metrics, address challenges like overfitting and underfitting, and understand the importance of feature selection and model tuning.

Description

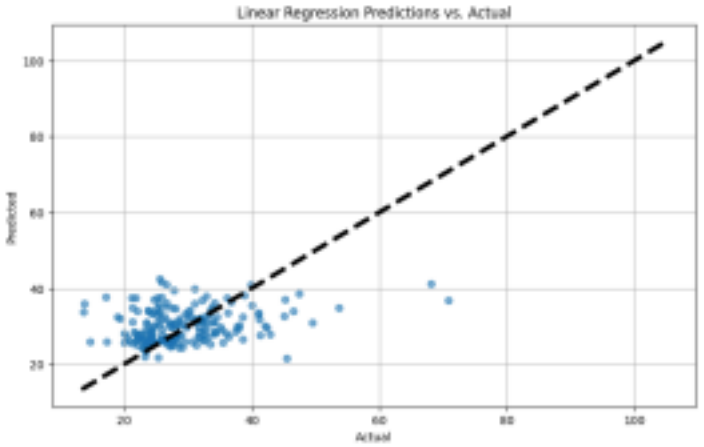
1. Classical ML: Supervised Learning (Classification and Regression).

a. What are the mean, median, mode, and standard deviation of the age, BMI, and Blood sodium columns in the dataset? Why are these statistics important for understanding the data?

b. How do the distributions of age, BMI, and Blood sodium look in the dataset? What can we learn from these distributions about the patient population?

c. Use pandas and scikit-learn to drop rows with missing values in the 'BMI' and 'Blood sodium' columns, and then uses logistic regression, SVM, kNN, and decision tree to predict an 'outcome' based on the features 'age', 'BMI', and 'blood sodium'. Ensure to split the data using *train_test_split* with a 20% test size and a random state of 42. Finally, fit the model, make predictions on the test set, and print a report of the best model ("classification_report"). Explain the result of the confusion matrix for the best model where you mention for each cell its meaning.

d. Predict BMI based on age and blood sodium with linear regression, SVM regressor, Decision tree regressor, and kNN regressor. Calculate RMSE, MSE, R-squared. Split where 20% left for the test, random state=42. Plot the actual vs. the predicted value for each model's test dataset. For example, the linear regression plot is:



2. Classical ML: Unsupervised Learning (Clustering, PCA)

a. Demonstrate the application of Principal Component Analysis (PCA) and t-Distributed Stochastic Neighbor Embedding (t-SNE) for dimensionality reduction on the dataset focusing on BMI, Blood sodium, and Blood calcium to visualize the data in a reduced-dimensional space. Compare the visualization results of PCA and t-SNE.

b. Apply K-means clustering to the dataset to group patients based on age, BMI, diabetes, and heart rate. Cluster to 2,3,4,5, and 6 groups. What are Silhouette and Davies-Bouldin Score for each case? What is the meaning of Davies-Bouldin value>1?

3. Deep Learning Principles

a. Describe the steps involved in training a neural network, including forward propagation and backpropagation.

b. Explain the bias-variance trade-off in neural network performance. How does it affect model generalization?

c. Highlight the importance of data preprocessing, normalization, and splitting for training effective deep learning models.

d. Train a DNN that classifies the outcome based on the age and the blood sodium only. For that you need to import several functions from *tensorflow.keras* ,ans *sklearn*, as described in this block:

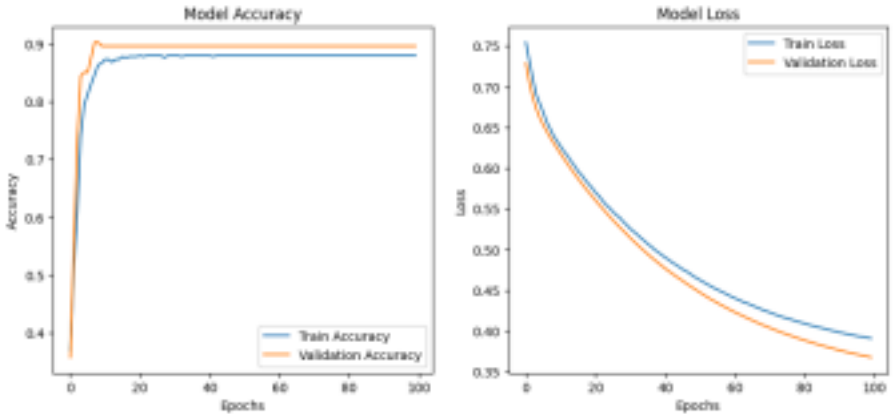
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.preprocessing import StandardScaler
```

i. Separate 'age' and 'blood sodium' as features (X) and 'outcome' as the label (y), ensure the data is clean by removing any rows with missing values using *dropna*(), and use *StandardScaler* from *sklearn.preprocessing* to standardize the features, ensuring that our model receives data within a normalized scale..

ii. *2.* Split the data into 4 sets: X_train, X_test, y_train, y_test. Use *train_test_split()* where train size is 80%. Keep the random state as 42.

iii. Construct a deep neural network using four dense layers. Configure the first three layers to each include 3 neurons and utilize the ReLU activation function. The final layer should contain a single neuron with a Sigmoid activation function, suitable for our binary classification tasks. Enhance the model's ability to generalize by incorporating two dropout layers with a dropout rate of 0.05, positioned between every two dense layers to reduce overfitting.

iv. Compile the neural network specifying *binary_crossentropy* as the loss function, adam as the optimizer, and accuracy as the performance metric. Train the model using a 20% validation split for 100 epochs to monitor and validate learning progress over time. Afterwards, graph both the training and validation accuracy, as well as the loss per epoch, to visually assess model performance and convergence. Use the provided examples as a guide for plotting.



v. Evaluate and Visualize Model Performance: Assess the model's accuracy on the test set to gauge its effectiveness in real-world scenarios. Next, visualize the results by plotting a confusion matrix. Additionally, display the model's architecture by invoking model.summary(), which provides a detailed overview of the model layers, their shapes, and the number of parameters involved.

vi. Design a deep neural network model tailored to our prediction task of in-hospital mortality. Select relevant features. Your model should include a combination of dense layers and activation functions optimized for binary classification. Try to play with the optimizer, loss, etc.